

Unit 4 Seminar – Programming Language Concepts and Security Implications

1. Recursion – Towers of Hanoi

The Towers of Hanoi requires transferring n disks between three pegs following strict rules: only one disk moves at a time, and no larger disk rests atop a smaller one (Cormen and Balkcom, n.d.). The recursive solution decomposes as: move $n-1$ disks to auxiliary, move n th disk to destination, move $n-1$ atop destination. This yields exactly $2^n - 1$ moves.

Security Analysis

Implementation faces two constraints:

1. Linear recursion depth ($n + \text{overhead}$), limited by Python's default recursion limit (typically 1000), triggering `RecursionError` beyond ≈ 990 .
2. Exponential runtime ($O(2^n)$ moves), e.g.:
 - $n = 20$: 1,048,575 moves
 - $n = 30$: 1,073,741,823 moves

User-controlled n transforms the algorithm into an availability threat: extreme CPU/time consumption, output flooding, resource exhaustion. This exemplifies algorithmic denial-of-service, where elegance masks latent risk.

Mitigations applied

- Strict input bounding (e.g., `MAX_DISKS = 20`).
- Recursion limit warnings.
- Defensive rejection of unsafe values.

SDLC Implication

Complexity must inform design/architecture phases. Secure SDLC demands explicit classification of algorithmic behaviour for user-influenced computations—recursion exemplifies how language constructs carry inherent security traits beyond injection risks.

2. Regex – UK Postcode Validation

UK postcodes comprise 5–7 alphanumeric characters (outward + inward, optional space), with structured formats: outward (2–4 chars, starts with letter); inward (exactly 3 chars, starts with digit) (Idealpostcodes, n.d.). Compliant regex must handle variations (e.g., A9 9AA to AA9A 9AA), GIR 0AA, and case-insensitivity.

Security Risks

Python's backtracking engine risks ReDoS via nested quantifiers, ambiguous alternations, or unbounded input (Weidman, n.d.; MITRE, 2024).

Mitigations

1. Length bound (≤ 8 chars) caps effort.
2. `fullmatch()` / anchoring (`^...$`) prevents partial paths.
3. Pattern avoids nested/unbounded repetition.

Critical Trade-Offs

Strict patterns reduce syntactic attack surface but risk false negatives on edge cases. Permissive ones enhance usability but permit bypasses. Regex provides syntax only—semantic validity requires authoritative lookup. Production adds rate limiting, monitoring, and potentially linear-time engines.

3. Integrated Security Reflection

Unit 4 reveals programming constructs harbour latent security properties:

- Recursion → exponential complexity → availability DoS under unbounded input.
- Regex → backtracking → ReDoS via adversarial strings.

Common traits:

- Vulnerabilities stem from design choices, not bugs.
- Risks manifest under extreme/adversarial conditions.
- Exploitation amplifies computation, not injects code.

Secure SDLC insight: security encompasses algorithm selection, complexity analysis, and engine behaviour. Unaware developers may embed DoS primitives in validation or recursion logic. Mitigation requires treating user-controlled iteration/recursion/pattern complexity as attack vectors, integrated via defence-in-depth.

Conclusion

These exercises show secure development transcends functional correctness: recursive purity and regex expressiveness can engender availability risks absent constraints. Unit 4 underscores controlling computational behaviour in worst-case scenarios through bounding, engine awareness, and layered validation—core to robust software security.

Word count: 748 (margin to typical 1000-word cap: 25%).

References

Cormen, T. H. and Balkcom, D. (n.d.) *Towers of Hanoi*. Available at: <https://www.khanacademy.org/computing/computer-science/algorithms/towers-of-hanoi> (Accessed: 20 February 2026).

Idealpostcodes (n.d.) *The UK Postcode Format*. Available at: <https://ideal-postcodes.co.uk/guides/uk-postcode-format> (Accessed: 20 February 2026).

MITRE (2024) 'CWE-1333: Inefficient Regular Expression Complexity'. Available at: <https://cwe.mitre.org/data/definitions/1333.html> (Accessed: 21 February 2026).

Weidman, A. (n.d.) 'Regular Expression Denial of Service – ReDoS', OWASP Foundation. Available at:

https://owasp.org/www-community/attacks/Regular_expression_Denial_of_Service_-_ReDoS (Accessed: 18 February 2026).